

**Name: Arava Raja**

**Reg No: 192365033**

**Course Code: CSA1718**

## **PROGRAM – 11**

### **Map Coloring**

#### **Aim**

To implement the **Map Coloring Problem** using **Constraint Satisfaction Problem (CSP)** in Python, such that no two adjacent regions are assigned the same color.

#### **Algorithm**

1. Define the map as a graph where each region is a node and edges represent neighboring regions.
2. Define a list of available colors.
3. Start with the first region.
4. Assign a color that does not conflict with the colors of its neighboring regions.
5. If no valid color is available, backtrack and try a different color for the previous region.
6. Repeat the process until all regions are assigned valid colors.
7. Display the color assigned to each region.

#### **Python Program**

```
# Map Coloring using CSP (Backtracking)  
# Graph representing neighboring regions
```

```
graph = {

    'A': ['B', 'C', 'D'],

    'B': ['A', 'C'],

    'C': ['A', 'B', 'D'],

    'D': ['A', 'C']

}

# Available colors

colors = ['Red', 'Green', 'Blue']


# Dictionary to store assigned colors

color_assignment = {}


# Function to check if color assignment is valid

def is_safe(region, color):

    for neighbor in graph[region]:

        if neighbor in color_assignment and color_assignment[neighbor] == color:

            return False

    return True


# Backtracking function
```

```
def map_coloring(regions):  
  
    if len(color_assignment) == len(regions):  
  
        return True
```

```
    region = regions[len(color_assignment)]
```

```
    for color in colors:
```

```
        if is_safe(region, color):
```

```
            color_assignment[region] = color
```

```
    if map_coloring(regions):
```

```
        return True
```

```
    del color_assignment[region]
```

```
    return False
```

```
regions = list(graph.keys())
```

```
if map_coloring(regions):
```

```
print("Color Assignment:")

for region in regions:

    print(region, "->", color_assignment[region])

else:

    print("No solution exists.")
```

## Expected Output

Color Assignment:

A -> Red

B -> Green

C -> Blue

D -> Green

# PROGRAM – 12

## Tic Tac Toe

### Aim

To implement the **Tic Tac Toe Game** in Python, allowing two players to play alternately on a **3 × 3** board until one player wins or the game ends in a draw.

### Algorithm

- ☐ Create a  $3 \times 3$  game board initialized with empty spaces.
- ☐ Display the board to the players. ☐ Set the first player as **X**.
- ☐ Repeat until the game ends:
  - Ask the current player to enter the row and column.
  - Check whether the selected position is empty.
  - If valid, place the player's symbol (**X** or **O**) on the board.
  - Display the updated board.
  - Check if the current player has won by completing a row, column, or diagonal.
  - If a player wins, display the winner and stop.
  - If all cells are filled and no winner exists, declare the game a draw.
  - Otherwise, switch to the other player.
- ☐ End the game.

# Python Program

```
# Tic Tac Toe Game
```

```
board = [[' ' for _ in range(3)] for _ in range(3)]
```

```
# Function to display the board
```

```
def display():
```

```
    for row in board:
```

```
        print(" | ".join(row))
```

```
    print("-" * 9)
```

```
# Function to check winnerdef check_winner(player):
```

```
    # Check rows
```

```
    for row in board:
```

```
        if all(cell == player for cell in row):
```

```
            return True
```

```
    # Check columns
```

```
    for col in range(3):
```

```
        if all(board[row][col] == player for row in range(3)):
```

```
            return True
```

```
    # Check diagonals
```

```
    if all(board[i][i] == player for i in range(3)):
```

```
        return True

    if all(board[i][2 - i] == player for i in range(3)):

        return True

    return False

# Function to check draw

def is_draw():

    for row in board:

        if ' ' in row:

            return False

    return True

# Main game loop

player = 'X'

while True:

    display()

    row = int(input(f'Player {player}, Enter row (0-2): '))

    col = int(input(f'Player {player}, Enter column (0-2): '))

    if board[row][col] == ' ':

        board[row][col] = player

        if check_winner(player):

            display()
```

```

        print(f'Player {player} Wins!')
        break
    if is_draw():
        display()
        print("Game Draw!")
        break
    player = 'O' if player == 'X' else 'X'
else:
    print("Cell already occupied! Try again.")

```

**Sample Input :** Player X, Enter row (0-2): 0  
 Player X, Enter column (0-2): 0

Player O, Enter row (0-2): 1  
 Player O, Enter column (0-2): 1

Player X, Enter row (0-2): 0  
 Player X, Enter column (0-2): 1

Player O, Enter row (0-2): 2  
 Player O, Enter column (0-2): 2

Player X, Enter row (0-2): 0  
 Player X, Enter column (0-2): 2

## Expected Output

```

X | X | X
-----
| O |
-----
| | O
-----

```

Player X Wins!



# PROGRAM – 13

## MIN – MAX Algorithm

### Aim :

To implement the **Minimax Algorithm** in Python to determine the optimal move for a player in a two-player game by maximizing the player's score and minimizing the opponent's score.

### Algorithm:

1. Represent the game tree with terminal node values.
2. Define a recursive **Minimax** function.
3. If the current node is a terminal node, return its value.
4. If it is the **Maximizer's** turn:
  - Initialize the best value to negative infinity.
  - Recursively evaluate all child nodes.
  - Return the maximum value.
5. If it is the **Minimizer's** turn:
  - Initialize the best value to positive infinity.
  - Recursively evaluate all child nodes.
  - Return the minimum value.
6. Continue until the root node is evaluated.
7. Display the optimal value obtained by the Maximizer.

### Program :

```
import math

# Minimax Function
def minimax(depth, nodeIndex, isMax, scores, height):

    # Base Case: Reached leaf node
    if depth == height:
        return scores[nodeIndex]

    if isMax:
```

```
        return max(
            minimax(depth + 1, nodeIndex * 2, False, scores, height),
            minimax(depth + 1, nodeIndex * 2 + 1, False, scores, height)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, scores, height),
            minimax(depth + 1, nodeIndex * 2 + 1, True, scores, height)
        )

# Leaf node values
scores = [3, 5, 2, 9, 12, 5, 23, 23]

# Height of the game tree
height = int(math.log2(len(scores)))

# Find the optimal value
optimal = minimax(0, 0, True, scores, height)

print("Optimal Value:", optimal)
```

**Expected Output :**

**Optimal Value: 12**